

00 - README - Como usar este TP de Prompting

TP — Aprender a Promptear con IA

Materia: DAI — Desarrollo de Aplicaciones I (ORT) **Modalidad:** Trabajo práctico incremental sobre un proyecto real
Proyecto base: API Express + PostgreSQL de `alumnos` / `cursos` (este repo) **Criterio de evaluación:** basado en la guía de *Uso de IA en EFSI* → <https://uso-de-ia-en-efsi.lovable.app/#builder>

De qué se trata este TP

No es un TP de "hacé que el código funcione". El código **ya funciona**.
Es un TP de **aprender a pedirle cosas a una IA y saber si te las dio bien**. La habilidad que vamos a evaluar no es "programar", es:

- 1. **Escribir un buen prompt** (con rol, contexto, tarea, restricciones e iteración).
- 2. **Leer críticamente lo que la IA te devuelve** y darte cuenta si está bien, mal o a medias.
- 3. **Iterar** — corregir el prompt cuando el resultado no sirve.
- 4. **Defender** lo que entregaste: si no entendés lo que la IA escribió, no lo entregues.

 **La trampa de este TP:** es facilísimo pedirle a una IA que "refactorice todo" y pegar el resultado. Lo difícil —y lo que se evalúa— es **demostrar que entendiste** qué pediste, por qué, y por qué el resultado está bien.

Qué podés hacer con IA en este TP (Semáforo EFSI)

Este TP **te obliga** a usar IA — esa es la idea. Pero hay reglas:

 Habilitado (libre)	 Con justificación	 Prohibido
Generar código de arranque (un CRUD nuevo, un middleware)	Resolver el ejercicio entero con un solo prompt gigante	Entregar código de IA sin proceso, reflexión ni comentarios
Pedir que te expliquen un error	Usar IA durante una evaluación presencial sin avisar	Compartir los prompts de evaluación en plataformas públicas
Pedir alternativas de diseño		Entregar algo que no podés explicar en el oral

● Habilitado (libre)	● Con justificación	● Prohibido
Preguntas conceptuales		
Generar datos de prueba		
Pedir code review de tu propio código		

⚠ **Trampa:** "lo hizo la IA" **no** es una justificación válida en el oral. Si la IA metió un bug y vos no lo viste, el bug es tuyo.

🧱 La estructura de un buen prompt (las 5 partes EFSI)

Todo prompt que entregues en este TP tiene que tener —idealmente— estas 5 piezas. No siempre las 5, pero cuanto más completo, mejor el resultado:

#	Parte	Qué es	Ejemplo aplicado a este proyecto
1	Rol	Qué experto querés que sea la IA	"Actuá como un desarrollador backend senior en Node.js y Express."
2	Contexto	Stack, estado actual, datos disponibles	"Tengo una API Express con arquitectura controller → service → repository, PostgreSQL con <code>pg</code> , ES modules. Te paso el archivo <code>alumnos-repository.js</code> ."
3	Tarea	El entregable exacto	"Quiero un repository nuevo para la tabla <code>materias</code> siguiendo el mismo patrón."
4	Restricciones	Límites de tecnología, formato, qué evitar	"No uses un ORM. Usá <code>pg</code> con queries parametrizadas <code>\$1</code> . Mantené el estilo de las clases existentes. No agregues dependencias nuevas."
5	Iteración	Cómo vas a refinar	"Después de verlo te voy a pedir que agregues manejo de errores y un filtro por nombre."

💡 **Tip:** Un prompt sin **contexto** y sin **restricciones** es el error #1 de los principiantes. La IA te va a dar algo genérico que no encaja con tu proyecto (te mete Sequelize, te cambia el estilo, te inventa carpetas). Pegá el código real y poné límites.

● Anti-ejemplo (prompt malo)

```
haceme el crud de materias
```

¿Qué le falta? Rol, contexto, restricciones, todo. La IA va a inventar un stack, probablemente no el tuyo.

● Ejemplo (prompt bueno)

Actuá como un desarrollador backend senior en Node.js.

Contexto: tengo una API REST en Express 5 con ES modules y arquitectura en capas (controller → service → repository). Uso la librería `pg` (no un ORM) con un Pool y queries parametrizadas con \$1, \$2. Te pego abajo mi `alumnos-repository.js` como referencia de estilo: [PEGÁS EL ARCHIVO]

Tarea: generá un `materias-repository.js` para una tabla `materias` (id, nombre, carga_horaria) con los métodos getAllAsync, getByIdAsync, createAsync, updateAsync y deleteByIdAsync, copiando exactamente el mismo patrón (clase que recibe una instancia de DbPg en `this.db` y ejecuta el SQL con this.db.queryAll/queryOne/queryReturnId/queryRowCount).

Restricciones: no agregues dependencias nuevas, no uses ORM, mantené los console.log de debug como en el original, y usá el operador ?? igual que en createAsync.

Después te voy a pedir el service y el controller, no los hagas todavía.

Los ejercicios (orden incremental)

Hacelos **en orden**. Cada uno asume que entendiste el anterior. Van de "generar código nuevo" (fácil) a "rediseñar arquitectura y seguridad" (difícil).

#	Ejercicio	Foco	Dificultad
01	Nueva tabla y su CRUD	Backend / generación guiada	★
02	Refactorización del CRUD repetido	DRY / refactorización	★ ★
03	Extracción de código repetido a Helpers	Modularización	★ ★
04	Validaciones y códigos de error	Validación / patrones	★ ★ ★
05	Middleware de Autenticación con JWT	Seguridad / auth	★ ★ ★
06	Arquitectura de la aplicación	Arquitectura	★ ★ ★ ★
07	Testing	Testing	★ ★ ★ ★
08	Performance y optimización	Performance	★ ★ ★ ★
09	Seguridad	Seguridad / auditoría	★ ★ ★ ★

💡 No hace falta hacer los 9 para aprobar. El docente te va a indicar cuáles son obligatorios y cuáles opcionales. Pero **todos** se entregan con la misma plantilla.

✅ Cómo se entrega cada ejercicio

Por **cada ejercicio** completás una copia de la [PLANTILLA - Bitácora de prompts y entrega](#). La entrega incluye (checklist EFSI):

1. **Repositorio GitHub** con commits ordenados (un commit por ejercicio, mensaje claro).
2. **Historial de la conversación con la IA** (PDF o link) — todos los prompts e iteraciones.
3. **Reflexión escrita** (300–600 palabras): qué pediste, qué decisiones tomaste, qué aprendiste, qué corregiste de la IA.
4. **Comentarios en el código** marcando qué generó la IA y qué modificaste vos. Por ejemplo:

```
// [IA] Generado con prompt #2 – ver bitácora ejercicio 01
// [YO] Cambié el ?? por validación explícita porque ...
```

5. **Defensa oral** (5–10 min): el docente te va a hacer preguntas sobre tu código y tus prompts.
-

🧠 Cómo te vamos a evaluar (resumen)

En criollo, evaluamos:

- **Calidad del prompt** — ¿tenía rol, contexto, tarea, restricciones, iteración?
- **Pensamiento crítico** — ¿detectaste cuando la IA se equivocó? ¿lo corregiste?
- **Comprensión** — ¿podés explicar qué hace el código que entregaste, línea por línea?
- **Proceso** — ¿iteraste o pegaste el primer resultado?
- **Honestidad** — ¿marcaste qué hizo la IA y qué hiciste vos?

🤖 Pregunta que te va a hacer el docente y conviene que sepas responder: "¿Qué fue lo más difícil de este ejercicio para vos, y cómo te diste cuenta de que la respuesta de la IA estaba bien?"